Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

# Department of CSE

B.Tech. CSE Trimester -5

## Roll Number: 2520030008
## Name: Sagi Sahasra Srihasa

# Generative AI
**25CS2207E**

**LAB RECORD**

**2026**

**Aim:**

To develop a provider-agnostic LLM client that supports both Gemini (cloud-based) and Ollama (locally hosted) models through a common interface, allowing users to switch between providers without changing the main application code.

**Code:**

**llm_client.py**

```
from abc import ABC, abstractmethod

from google import genai

import requests



# Common interface

class LLMProvider(ABC):


    @abstractmethod

    def generate(self, prompt):

        pass



# Gemini cloud provider

class GeminiProvider(LLMProvider):
```

```
    def __init__(self, api_key):
        self.client = genai.Client(api_key=api_key)

    def generate(self, prompt):
        response = self.client.models.generate_content(
            model="gemini-3.8-flash",
            contents=prompt
        )
        return response.text


# Ollama local provider
class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2:3b"):
        self.model = model
        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
        response = requests.post(
            self.url,
```

```
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=120
        )

        response.raise_for_status()
        return response.json()["response"]


# Common client
class LLMClient:

    def __init__(self, provider):
        self.provider = provider

    def generate(self, prompt):
        return self.provider.generate(prompt)
```

**main.py:**

```
import os
from dotenv import load_dotenv

from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)

load_dotenv(override=True)

provider_name = input(
    "Choose provider (gemini/ollama): "
).strip().lower()

if provider_name == "gemini":

    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        raise ValueError("Gemini API key not found in .env")
```

```
    provider = GeminiProvider(api_key)

elif provider_name == "ollama":

    provider = OllamaProvider("llama3.2:3b")

else:
    raise ValueError("Invalid provider selected")

client = LLMClient(provider)

prompt = input("Enter your question: ")

answer = client.generate(prompt)

print("\nLLM Response:")
print(answer)
```

**Test_client.py :**

```
import unittest
from llm_client import LLMClient



# Fake provider for testing
class FakeProvider:

    def generate(self, prompt):
        return "AI stands for Artificial Intelligence."



class TestLLMClient(unittest.TestCase):

    def test_generate(self):

        provider = FakeProvider()
        client = LLMClient(provider)

        result = client.generate("Explain AI")

        self.assertEqual(
```

```
            result,
            "AI stands for Artificial Intelligence."
        )



if __name__ == "__main__":
    unittest.main()
```

**.env:**

```
GEMINI_API_KEY=your_gemini_api_key
```

**Requirements.txt:**

```
python-dotenv
pytest
google-genai
requests
```

## Output:

```
C:\Users\sahas\OneDrive\Desktop\Btech\GenAI\Lab\Week2\.venv\Scripts\python.exe C:\Users\
Choose provider (gemini/ollama): ollama
Enter your question: what is ai? explain in one sentence.

LLM Response:
Artificial Intelligence (AI) refers to the development of computer systems that can per

Process finished with exit code 0
```